

PROYECTO FINAL 1

Sistema de Gestión de Inventarios Empresarial. Curso FRONTEND II

Alumno: JOHN FRADER ARRUBLA LOPEZ cc. 9725307 cel. 310 467 0681

Objetivo General:

Desarrollar una aplicación web que simule la gestión de inventario de una empresa, permitiendo controlar productos, movimientos y reportes usando HTML, CSS Framework, JavaScript y localStorage.

Nivel:

Últimos semestres – Proyecto de nivel profesional (sin backend).

DESARROLLO**PLAINTEXT:**

[Fase 1: Base de Datos Local]

—> [Fase 2: Autenticación] —> [Fase 3: Lógica y CRUD]

—> [Fase 4: UI y Reportes]

Fase 1: Estructura de Datos y Persistencia**Prompt 1 (Estructura de Datos):**

"Hola! Necesito organizar la información para un proyecto de inventario en JavaScript sin usar base de datos, solo localStorage. Tengo que guardar usuarios (admin y empleado), productos, categorías y un historial de movimientos (entradas y salidas de stock). ¿Me podrías mostrar un ejemplo de cómo estructurar esos objetos en JSON para que no sea complicado relacionarlos después?"

Prompt 2 (Datos Semilla / Inicialización):

"Tengo un problema: cada vez que abro mi página en blanco, el localStorage está vacío. ¿Cómo puedo hacer un script sencillo en JavaScript que revise si ya hay datos guardados, y si no hay nada, cargue automáticamente unos productos y usuarios de prueba para poder trabajar?"

Fase 2: Control de Accesos y Permisos

Prompt 3 (Manejo de Roles):

"Estoy armando la pantalla de inicio de sesión. Guardé el usuario actual en el localStorage con su rol ("ADMIN" o "EMPLOYEE"). ¿Cómo hago un código en JS para que, si entra un empleado, se oculten automáticamente los botones de eliminar y editar productos en la página?"

Prompt 4 (Protección de Rutas):

"¿Cómo puedo evitar que un usuario entre directo a la página dashboard.html escribiendo la URL en el navegador si no ha iniciado sesión antes? Necesito que lo regrese al index.html."

Fase 3: Lógica de Inventario y Movimientos

Prompt 5 (Lógica de Stock):

"Necesito una función en JavaScript para registrar cuando entra o sale mercancía. Si es una salida, debe revisar que la cantidad a sacar no sea mayor al stock disponible para que no quede en números negativos. Si hay suficiente stock, debe descontarlo del producto y agregar un registro con la fecha al historial de movimientos."

Prompt 6 (Corrección de Error / Depuración):

"Cuando edito un producto desde un formulario en HTML, se me están borrando los datos que no edité (por ejemplo, el historial o la categoría). ¿Cómo actualizo solo el nombre y el precio de un objeto en JS sin sobrescribir todo lo demás?"

Fase 4: Métricas, Interfaz y Experiencia de Usuario

Prompt 7 (Panel de Estadísticas):

"Para el rol de administrador me piden mostrar alertas de productos con stock bajo. ¿Cómo hago en JS para filtrar la lista de productos y extraer solo los que tengan menos de 5 unidades, y además saber cuál es el producto que más movimientos ha tenido?"

Prompt 8 (Mejora de UI/Tabla):

"Tengo una tabla en HTML donde muestro los productos con Vanilla JS. ¿Cómo le puedo aplicar clases de Bootstrap para que si un producto tiene menos de 5 unidades, su fila o una etiqueta se ponga de color rojo, y si tiene más, de color verde?"

Siguiente Paso

Implementación del código paso a paso:

Módulo 1: storage.js.

Este archivo es el corazón del sistema al trabajar sin backend, pues encapsula la interacción con localStorage, inicializa los datos iniciales (semilla) si el navegador está limpio y provee funciones sencillas de CRUD para usar en el resto de la aplicación.

Código: [storage.js](#)

Breve explicación técnica

- **Puntos clave:** Usamos `JSON.parse()` para convertir el texto guardado en localStorage a un objeto o arreglo JS interactivo, y `JSON.stringify()` para volverlo a texto antes de guardar.
- **Técnica limpia:** La función `saveProduct` usa el operador spread (`...products[index], ...product`), lo cual resuelve directamente lo planteado en el **Prompt 6** (evitar que se borren datos al editar un producto).

Módulo 2: auth.js.

Este módulo se encarga de la simulación del Login, el Logout y la Protección de Rutas y Elementos del DOM según el rol (ADMIN vs EMPLOYEE). Con este archivo cumplimos los requerimientos de "Protección de vistas según rol" y la restricción de que los empleados no puedan eliminar ni editar productos.

Código: [auth.js](#)

Breve explicación técnica

- **Atributo `data-role="ADMIN"`:** En lugar de llenar el código JS con IDs específicos para ocultar botones uno a uno, usamos un atributo HTML personalizado (`data-role="ADMIN"`). Cualquier botón, enlace o sección en el HTML que tenga ese atributo se ocultará automáticamente si la persona logueada es un empleado.
- **Seguridad en cliente:** Explicar al evaluador que en una arquitectura Frontend Pura (sin backend), el bloqueo del DOM es visual para la experiencia de usuario, y la validación lógica de acciones (como borrar un producto) también se rectifica en las funciones de negocio.

Módulo 3: movements.js.

Este archivo contiene la lógica core del negocio: el control de entradas (compras) y salidas (ventas/mermas) de productos, la actualización automática del inventario en localStorage y las consultas requeridas para las estadísticas del Administrador (alertas de stock bajo y productos más movidos).

Código: [movements.js](#)

Puntos clave para defender en la sustentación

1. **Atomicidad lógica (Prompt 3 y 5):** La validación de disponibilidad (`product.stock < qty`) ocurre **antes** de mutar el objeto en memoria y de escribir en localStorage. Si la condición falla, la función retorna de inmediato sin alterar ningún dato.
2. **Formato ISO 8601:** Las fechas de los movimientos se guardan con `new Date().toISOString()`. Esto es una buena práctica Senior porque permite formatear la fecha según el idioma o zona horaria local del cliente sin perder precisión.
3. **Optimización con Enriquecimiento en Caliente:** Para evitar duplicar datos (como guardar el nombre completo del producto dentro de cada registro de movimiento), guardamos solo `productId` y usamos `getEnrichedMovements()` para unir (JOIN) las entidades al momento de renderizar la UI.

Módulo 4: index.html.

Está diseñado con Tailwind CSS para un acabado moderno y profesional. Incluye un selector de usuarios de prueba (Admin y Empleado) para probar el sistema al instante, así como un campo de texto por si se prefiere escribir el nombre de usuario manualmente.

Código: [index.html](#)

Detalles técnicos destacables

1. **Auto-redirección si ya está logueado:** Si el usuario ya tiene sesión iniciada en localStorage, la página no muestra el formulario y lo envía directo a `dashboard.html`.
2. **Sincronización interactiva:** Al seleccionar una opción en el `<select>`, el campo `<input>` se autorrellena solo, facilitando las pruebas tanto para ti como para el profesor evaluador.
3. **Manejo de errores amigable:** Si el usuario escribe cualquier texto inventado, una alerta de Tailwind en color rojo le recuerda los usuarios disponibles.

Módulo 5: dashboard.html.

Aquí tienes la estructura HTML completa de **dashboard.html** utilizando **Tailwind CSS**.

Incluye:

- **Barra de navegación:** Nombre del sistema, perfil del usuario logueado y botón de Logout.
- **Panel de Estadísticas (Exclusivo ADMIN):** Indicadores visuales para productos con stock bajo y producto con más movimientos.
- **Sección de Productos:** Filtros en tiempo real (búsqueda por texto y categoría), botón para **Nuevo Producto** (exclusivo ADMIN) y la tabla interactiva de inventario con identificadores por colores (badges de stock).
- **Sección de Historial de Movimientos:** Tabla detallada de entradas y salidas con fecha formateada.
- **Modales:**
 1. Modal de Producto (Crear/Editar).
 2. Modal de Movimientos (Registrar Entradas / Salidas de Stock).
 3. Modal de Categorías (Crear nuevas categorías).

Código: [dashboard.html](#)

Módulo 6: dashboard.html.

Aquí tienes el archivo **js/app.js** completo. Este script actúa como el **controlador principal (UI Controller)**: conecta los datos almacenados en localStorage con la interfaz gráfica, gestiona los filtros en tiempo real, abre y cierra los modales, y aplica la protección de acciones según el rol del usuario.

Código: [app.js](#)

Resumen de la Arquitectura Implementada

- **Separación de responsabilidades:**
 - storage.js: Acceso directo a localStorage y datos semilla.
 - auth.js: Autenticación, sesión y guards por rol.
 - movements.js: Lógica de negocio (control estricto de stock y métricas).
 - app.js: Interacción con el usuario, eventos y modales.
- **Flujo 100% funcional:** La aplicación ya no depende de ningún servidor ni librería externa pesada, cumpliendo al 100% con los criterios de evaluación.